

Homework 2

2.50.)

a.

We can find T by finding a few elements of y in terms of elements u and finding a pattern.

$$y(0) = \dots + h_2u(-2) + h_1u(-1) + h_0u(0) + h_{-1}u(1) + h_{-2}u(2) + \dots$$

We know that $u(k) = 0$ for $k < 0$ and $h_j = 0$ for $j < 0$, so this reduces to

$$y(0) = h_0u(0)$$

Similarly

$$y(1) = \dots + h_2u(-1) + h_1u(0) + h_0u(1) + h_{-1}u(2) + h_{-2}u(3) + \dots$$

reduces to

$$y(1) = h_1u(0) + h_0u(1)$$

In other words, we can use $u(k) = 0$ for $k < 0$ and $h_j = 0$ for $j < 0$ to take out zero elements and simplify our general formula for y to be

$$y(k) = \sum_{j=0}^k h_j u(k-j) = h_k u(0) + h_{k-1} u(1) + \dots + h_0 u(k)$$

This can then be represented with the Toeplitz matrix T :

$$T = \begin{bmatrix} h_0 & 0 & 0 & \dots & 0 \\ h_1 & h_0 & 0 & \dots & 0 \\ h_2 & h_1 & h_0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ h_N & h_{N-1} & h_{N-2} & \dots & h_0 \end{bmatrix}$$

b.

We can find H by finding a few elements of y in terms of elements u and finding a pattern.

$$y(0) = \dots + h_2u(-2) + h_1u(-1) + h_0u(0) + h_{-1}u(1) + h_{-2}u(2) + \dots$$

We know that $u(k) = 0$ for $k > 0$ or $k < -N$ and $h_j = 0$ for $j < 0$, so this reduces to

$$y(0) = h_N u(-N) + h_{N-1} u(-N+1) + \dots + h_1 u(-1) + h_0 u(0)$$

Similarly

$$y(1) = \dots + h_3u(-2) + h_2u(-1) + h_1u(0) + h_0u(1) + h_{-1}u(2) + \dots$$

reduces to

$$y(1) = h_{N+1} u(-N) + h_N u(-N+1) + \dots + h_2 u(-1) + h_1 u(0)$$

In other words, we can use $u(k) = 0$ for $k > 0$ or $k < -N$ and $h_j = 0$ for $j < 0$ to take out zero elements and simplify our general formula for y to be

$$y(k) = \sum_{j=k}^{k+N} h_j u(k-j) = h_k u(0) + h_{k+1} u(-1) + \dots + h_{k+N} u(-N)$$

This can then be represented with the Hankel matrix H :

$$H = \begin{bmatrix} h_0 & h_1 & h_2 & \dots & h_N \\ h_1 & h_2 & h_3 & \dots & h_{N+1} \\ h_2 & h_3 & h_4 & \dots & h_{N+2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ h_N & h_{N+1} & h_{N+2} & \dots & h_{N+N} \end{bmatrix}$$

4.590.)

We know that $\tilde{a} = \frac{a}{\|a\|}$ is the unit vector in the direction of a . We can then take $\tilde{a}$ and extend it to be an orthonormal basis for $\mathbb{R}^n$ like so

$$Q = [\tilde{a} \quad Q_2]$$

This would make Q orthogonal in $\mathbb{R}^n$ and Q_2 be the orthonormal basis for the hyperplane $\{z : a^\top z = 0\}$. We then have

$$x = \tilde{a}\tilde{a}^\top x + Q_2Q_2^\top x$$

This splits up x into the projection of x onto the hyperplane, $Q_2Q_2^\top x$, and the projection of x onto a , $\tilde{a}\tilde{a}^\top x$. We find the reflection of x , call it y , by taking the projection of x onto the hyperplane and reversing the projection of x onto a . In other words,

$$y = -\tilde{a}\tilde{a}^\top x + Q_2Q_2^\top x = x - 2\tilde{a}\tilde{a}^\top x = (I_n - 2\tilde{a}\tilde{a}^\top)x$$

Thus, we have

$$R = I_n - 2\tilde{a}\tilde{a}^\top$$

Now we just have to verify R is orthogonal. We know that $R \in \mathbb{R}^{n \times n}$, so it's square. We also have

$$R^\top R = (I_n - 2\tilde{a}\tilde{a}^\top)^\top (I_n - 2\tilde{a}\tilde{a}^\top) = (I_n - 2\tilde{a}\tilde{a}^\top)(I_n - 2\tilde{a}\tilde{a}^\top)$$

*We can get rid of the transpose because $I_n = I_n^\top$ and $\tilde{a}\tilde{a}^\top = (\tilde{a}\tilde{a}^\top)^\top$

$$= I_n I_n - 2I_n \tilde{a}\tilde{a}^\top - 2\tilde{a}\tilde{a}^\top I_n + 4\tilde{a}\tilde{a}^\top \tilde{a}\tilde{a}^\top = I_n - 2\tilde{a}\tilde{a}^\top - 2\tilde{a}\tilde{a}^\top + 4\tilde{a}\tilde{a}^\top = I_n$$

Hence, R is square and $R^\top R = I_n$, so R is orthogonal.

4.600.)

We have that y is consistent is equivalent to $y = Ax$ for some $x \in \mathbb{R}^n$ which is equivalent to $y \in \text{range}(A)$. Note that $By = 0$ is equivalent to $y \in \text{null}(B)$. Using these equivalences, the question is thus asking us to construct B such that $y \in \text{range}(A)$ if and only if $y \in \text{null}(B)$. Take a full QR factorization of A :

$$A = QR = [Q_1 \quad Q_2] \begin{bmatrix} R_1 \\ 0 \end{bmatrix}$$

Where Q_1 is the orthonormal basis for $\text{range}(A)$ and Q_2 is the orthonormal basis for $\text{range}(A)^\perp$. We'll show that $y \in \text{range}(A)$ if and only if $y \in \text{null}(B)$ holds for $B = Q_2^\top$. From our QR decomposition, we have for any y , $y = Q_1Q_1^\top y + Q_2Q_2^\top y$. As Q_1 is the orthonormal basis for $\text{range}(A)$, we have

$$y \in \text{range}(A) = \text{range}(Q_1) \Leftrightarrow y = Q_1Q_1^\top y \Leftrightarrow Q_2Q_2^\top y = 0 \Leftrightarrow y \in \text{null}(Q_2^\top) = \text{null}(B)$$

Thus $B = Q_2^\top$ holds under our desired conditions, making it a good integrity monitor. As can be seen in the Julia

code below, we found that for $A = \begin{bmatrix} 1 & 2 & 1 \\ 1 & -1 & -2 \\ -2 & 1 & 3 \\ 1 & -1 & -2 \\ 1 & 1 & 0 \end{bmatrix}$ that

$$B = \begin{bmatrix} -0.0724 & -0.8642 & -0.4639 & 0.1322 & -0.1234 \\ 0.0894 & -0.1348 & 0.4592 & 0.8683 & 0.0955 \\ -0.5367 & -0.1129 & 0.0503 & -0.0801 & 0.8306 \end{bmatrix}$$

```

1  using LinearAlgebra
2  using Plots
3
4  include("qr.jl")
5
6  A = [1 2 1
7       1 -1 -2
8       -2 1 3
9       1 -1 -2
10      1 1 0]
11
12  Q, R = fullqr(A)
13
14  println()
15  println("Rank of R is ", rank(R), ", so Q2 has ", rank(Q) - rank(R), " basis vectors")
16  println()
17
18  Q2 = Q[:, 3:5]
19  B = Q2'
20
21  println("B = ", B)
22  println()
23  println("BA = ", B * A)
24  println()
25  println("Bq3 = ", B * Q[:, 3])
26  println()
27  println("Bq4 = ", B * Q[:, 4])
28  println()
29  println("Bq5 = ", B * Q[:, 5])
30

```

Rank of R is 2, so Q2 has 3 basis vectors

B = [-0.07236941073736788 -0.8642122912401964 -0.46389864920408886 0.13217961206539075 -0.12339520849598737; 0.08940363712636537 -0.13479968104534637 0.45917366281369565 0.8682619938290725 0.09548137571729959; -0.5369887891245073 -0.11288905809342586 0.05028935490053818 -0.08017130411802041 0.8306278611370306]

BA = [8.049116928532385e-16 1.5820678100900848e-15 8.881784197001252e-16; -1.6653345369377348e-16 -1.6653345369377348e-16 0.0; 5.551115123125783e-16 4.440892098500626e-16 -2.220446049258313e-16]

Bq3 = [1.0000000000000002, 4.863202114447188e-17, -1.0716566586221212e-16]

Bq4 = [4.863202114447188e-17, 0.9999999999999999, 2.5201161886159222e-17]

Bq5 = [-1.0716566586221212e-16, 2.5201161886159222e-17, 1.0]

To explain the code a bit, we first took the full QR decomposition of A and looked at the difference of the ranks of Q and R to see how many basis vectors there were for Q_2 . As this difference was three, that meant Q_2 is the last three columns of Q (apparently the third column of A is linearly dependent of the first two), which then gives us B . Note that k , the number of rows in B , must be at least the difference of ranks of Q and R (three in this case) as that's the dimension of $\text{range}(A)^\perp$ and we need to ensure every vector in $\text{range}(A)^\perp$ isn't in the null space of B by having them all be in the row space of B instead. After that we just verified B . We did this by showing that $BA = 0$ and $Bq_3 = e_1, Bq_4 = e_2, Bq_5 = e_3$. $BA = 0$ implies $\text{range}(A) \subseteq \text{null}(B)$. By the QR decomposition, any vector outside the range of A has a nonzero projection on at least one of q_3, q_4, q_5 , so if B maps all of them to nonzero linearly independent vectors (l.i. so they can't cancel each other out), then any vector outside the range of A must map to a nonzero vector as well. In other words, this implies $y \notin \text{range}(A) \Rightarrow y \notin \text{null}(B)$. Combined, we have $\text{range}(A) = \text{null}(B)$. These can all be seen in the code output, so we're done.

4.630.)

a, e, i
b, g, k
c
d, h
f, j

6.741.)

This problem can be solved using least squares estimation.

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} = \begin{bmatrix} l_{11} & l_{21} & \dots & l_{n^2 1} \\ l_{12} & l_{22} & \dots & l_{n^2 2} \\ \vdots & \vdots & \ddots & \vdots \\ l_{1N} & l_{2N} & \dots & l_{n^2 N} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n^2} \end{bmatrix} + \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_N \end{bmatrix}$$

$$y = Lx + v \Rightarrow \tilde{x} = (L^\top L)^{-1} L^\top y$$

Note that this works because $N = 1225 > n^2 = 900$ which makes L a skinny matrix and the equation above for the pseudo inverse of L hold. In the Julia code, we take the data vector and reshape and transpose it to get into the format in the equation above. We then find the pseudo inverse, find the least squares estimation and reshape that into a square matrix to get our picture at the end.

```
31 | #Question 5.)
32 |
33 | #Read Data from JSON file
34 |
35 | include("readclassjson.jl")
36 | data = readclassjson("tomo_data.json")
37 |
38 | y = data["y"]
39 | npixels = data["npixels"]
40 | N = data["N"]
41 | line_pixels_length = data["line_pixel_lengths"]
42 |
43 | L = reshape(line_pixels_length, npixels*npixels, N)
44 | L = L'
45 | pseudo_L_inverse = inv(L' * L) * L'
46 | ls_x = pseudo_L_inverse * y
47 | square_ls_x = reshape(ls_x, npixels, npixels)
48 | heatmap(square_ls_x, yflip=true, aspect_ratio=:equal, color=:gist_gray, cbar=:none, framestyle=:none)
```

